REVISION CHEAT SHEET:

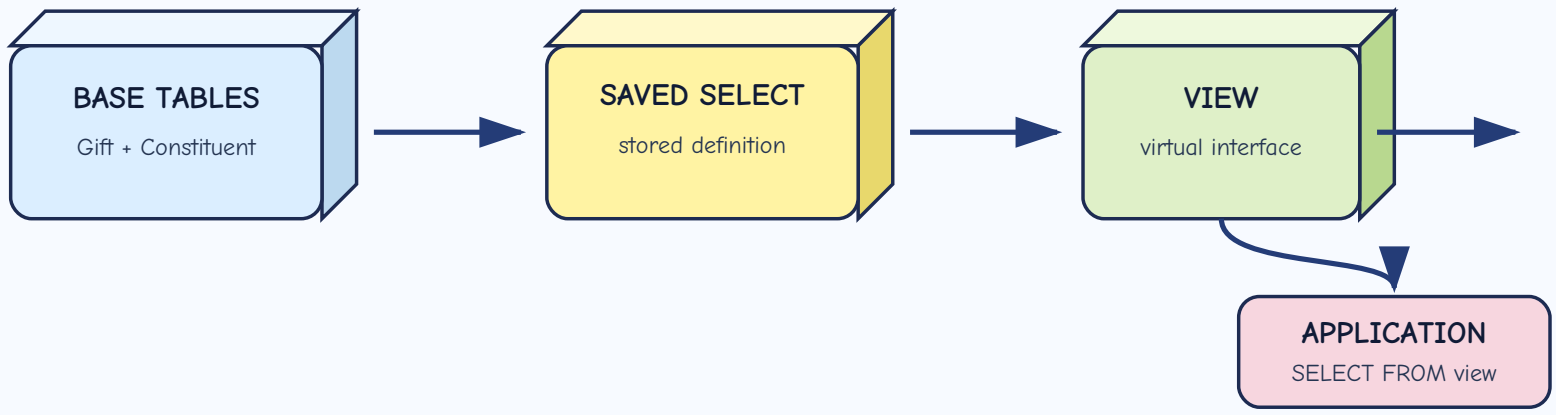
SQL Server Views

INTERVIEW-READY NOTES - NORMAL VIEWS - SECURITY - UPDATABLE VIEWS - INDEXED VIEWS

3D DIAGRAM

Normal View Flow

A normal view stores the SELECT definition, not a separate copy of result rows.



Base tables remain authoritative; the saved view definition provides a reusable relational interface.

TOPIC & SUBTOPIC INDEX

Study order: view basics -> create/query/alter/drop -> metadata/dependencies -> view types -> updates/security -> comparisons -> performance/indexed views -> practical views -> interview revision.

What Is a View?	First FundraisingDb Example	Why Use Views?
View Definition	Basic CREATE VIEW Syntax	Querying a View
ALTER VIEW	DROP VIEW	View Metadata
View Dependencies	Types of Views	Simple View
Filtered View	Joined View	Aggregate View
View with GROUP BY	View with HAVING	UNION View
View with Subquery	View with ORDER BY	Updatable Views
WITH CHECK OPTION	Security Through Views	View vs Table
View vs Temporary Table	View vs CTE	View vs Stored Procedure
Nested Views	View Performance	WITH SCHEMABINDING
WITH ENCRYPTION	Indexed Views	Indexed View Creation Requirements
Indexed View Restrictions	Indexed View Maintenance Cost	sp_refreshview
Security Best Practices with Views	Common View Mistakes	Practical View 1 — Donor Gift Summary
Choosing a View	Master View Comparison	25 Interview Questions and Practical Problems
Final Revision Diagram	Final Easy Memory Rules	

A **view** is a named SQL query stored in the database.

A normal view usually does **not store its own copy of the result rows**.

Instead, it stores the query definition.

When you query the view, SQL Server uses that definition to obtain data from the underlying tables.

Think:

EXAMPLE / FLOW

```
Base tables
     ↓
Stored SELECT definition
     ↓
View
     ↓
Application queries the view
```

Example:

EXAMPLE / FLOW

```
constituent.Constituent
        +
fundraising.Gift
        ↓
vw_ConstituentGiving
```

The application can query:

SQL

```sql
SELECT *
FROM fundraising.vw_ConstituentGiving;
```

instead of rewriting the full underlying query every time.

What Is a View?

Simple Definition

A **view** is a virtual table based on a saved SELECT query.

It can expose:

EXAMPLE / FLOW

```
columns from one table,

columns from several joined tables,

calculated values,

filtered rows,

aggregated results,

or combined query results.
```

For a normal view, the underlying tables remain the actual source of the data.

First FundraisingDb Example

Suppose we frequently need:

Gift information together with the donor's display name.

Without a view:

SQL

```sql
SELECT
    g.GiftId,
    g.GiftDate,
    g.TotalAmount,
    c.ConstituentId,
```

```
    c.DisplayName
FROM fundraising.Gift AS g
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId;
```

Your database already has the relationship:

EXAMPLE / FLOW

```
Constituent
    1
    |
    | many
    v
Gift
```

We can store that query as a view.

SQL

```
CREATE VIEW fundraising.vw_GiftDonor
AS

SELECT
    g.GiftId,
    g.GiftDate,
    g.TotalAmount,
    g.CampaignId,
    c.ConstituentId,
    c.DisplayName

FROM fundraising.Gift AS g

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId;
GO
```

Now:

SQL

```
SELECT *
FROM fundraising.vw_GiftDonor;
```

returns the joined result.

View Flow

EXAMPLE / FLOW

```
fundraising.Gift
        +
constituent.Constituent
        |
        v
CREATE VIEW
vw_GiftDonor
        |
        v
Application
```

Why Use Views?

Views are commonly used for:

EXAMPLE / FLOW

simplifying complex queries,

reusing common query logic,

hiding unnecessary columns,

providing controlled data access,

presenting business-friendly data,

abstracting underlying table structure,

aggregating data,

standardizing joins.

Example — Simplifying Application Queries

Without a view:

SQL

```
SELECT
    c.DisplayName,
    g.GiftDate,
    g.TotalAmount,
    ca.CampaignName

FROM constituent.Constituent AS c

INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId

LEFT JOIN fundraising.Campaign AS ca
    ON ca.CampaignId = g.CampaignId;
```

With a view:

SQL

```
SELECT *
FROM fundraising.vw_GiftSummary;
```

The application does not need to repeat the joins.

View Definition

The query after:

SQL

```
AS
```

is the **view definition**.

Example:

SQL

```
CREATE VIEW fundraising.vw_LargeGifts
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

The stored definition is conceptually:

EXAMPLE / FLOW

```
SELECT
GiftId,
ConstituentId,
GiftDate,
TotalAmount
FROM Gift
WHERE TotalAmount >= 1000
```

Basic CREATE VIEW Syntax

SQL

```
CREATE VIEW schema_name.view_name
AS

SELECT
    ...
FROM ...;
GO
```

Example:

SQL

```
CREATE VIEW fundraising.vw_LargeGifts
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
GO
```

Why Use a Schema for Views?

Just like tables, views belong to schemas.

Examples:

EXAMPLE / FLOW

```
fundraising.vw_GiftDonor

constituent.vw_ConstituentContact

engagement.vw_OpenTasks
```

Using schema-qualified names makes ownership and purpose clearer.

Querying a View

A view is queried much like a table.

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.vw_LargeGifts;
```

You can add additional filtering:

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.vw_LargeGifts
WHERE GiftDate >= '2026-01-01';
```

ALTER VIEW

Simple Definition

`ALTER VIEW` changes the definition of an existing view without dropping and recreating the view.

Example original view:

```sql
CREATE VIEW fundraising.vw_LargeGifts
AS

SELECT
    GiftId,
    ConstituentId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
GO
```

Suppose we also want:

```
EXAMPLE / FLOW
GiftDate
```

Use:

```sql
ALTER VIEW fundraising.vw_LargeGifts
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
GO
```

Microsoft SQL Server supports changing a view with `ALTER VIEW`; dependent objects remain, although changing the definition can make those dependencies invalid if they relied on the previous shape.

Important ALTER VIEW Rule

If the old view used options such as:

```
EXAMPLE / FLOW
WITH ENCRYPTION

WITH CHECK OPTION
```

you must include them again in the `ALTER VIEW` statement if you want them to remain enabled.

DROP VIEW

Simple Definition

`DROP VIEW` removes the view object from the database.

It does **not** delete data from the underlying tables.

Example:

SQL

```
DROP VIEW fundraising.vw_LargeGifts;
GO
```

After this:

EXAMPLE / FLOW

```
vw_LargeGifts
```

no longer exists.

But:

EXAMPLE / FLOW

```
fundraising.Gift
```

is unchanged.

View vs Table When Dropping

EXAMPLE / FLOW

```
DROP VIEW
→ Removes query definition
```

EXAMPLE / FLOW

```
DROP TABLE
→ Removes table structure/data object
```

These are completely different operations.

View Metadata

SQL Server stores metadata about views in system catalog views.

The most useful include:

EXAMPLE / FLOW

```
sys.views

sys.columns

sys.sql_modules

sys.sql_expression_dependencies
```

`sys.views` contains one row for every view object in the database.

Find All Views

SQL

```
SELECT
    SCHEMA_NAME(schema_id) AS SchemaName,
    name AS ViewName
FROM sys.views
ORDER BY
    SchemaName,
    ViewName;
```

Find View Columns

SQL

```
SELECT
    c.column_id,
    c.name AS ColumnName,
    TYPE_NAME(c.user_type_id) AS DataType,
    c.max_length,
    c.is_nullable

FROM sys.columns AS c

WHERE c.object_id =
      OBJECT_ID
      (
          'fundraising.vw_GiftDonor'
      )

ORDER BY c.column_id;
```

View the Stored Definition

One option:

SQL

```
SELECT
    OBJECT_DEFINITION
    (
        OBJECT_ID
        (
            'fundraising.vw_GiftDonor'
        )
    ) AS ViewDefinition;
```

Another:

SQL

```
EXEC sys.sp_helptext
    N'fundraising.vw_GiftDonor';
```

`sp_helptext` can display stored object-definition text when the caller has the required metadata visibility.

View Dependencies

Simple Definition

A **dependency** exists when one database object references another.

Example:

EXAMPLE / FLOW

```
vw_GiftDonor
    |
    +---- depends on fundraising.Gift
    |
    +---- depends on constituent.Constituent
```

If one of those tables changes, the view may be affected.

Find Objects Referenced by a View

Use:

SQL

```
SELECT
    referenced_schema_name,
    referenced_entity_name,
    referenced_minor_name
FROM sys.sql_expression_dependencies
WHERE referencing_id =
```

```
      OBJECT_ID
      (
          'fundraising.vw_GiftDonor'
      );
```

`sys.sql_expression_dependencies` is the modern catalog view for tracking by-name dependencies between SQL objects.

Find Views That Depend on Gift

SQL

```
SELECT
    OBJECT_SCHEMA_NAME(referencing_id)
        AS ReferencingSchema,

    OBJECT_NAME(referencing_id)
        AS ReferencingObject

FROM sys.sql_expression_dependencies

WHERE referenced_id =
      OBJECT_ID
      (
          'fundraising.Gift'
      );
```

Do Not Use sp_depends for New Development

Older SQL Server code may use:

SQL

```
sp_depends
```

Microsoft has marked this for future removal and recommends newer dependency tools such as:

EXAMPLE / FLOW

```
sys.sql_expression_dependencies

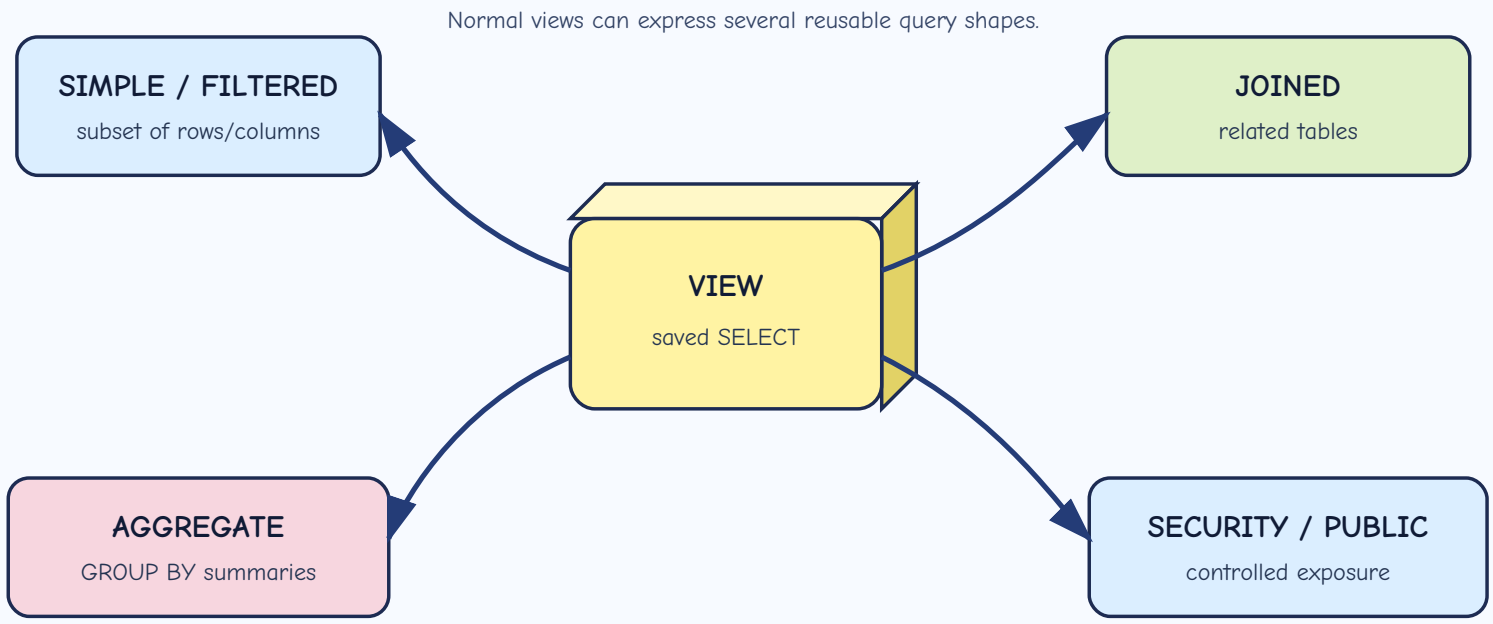
sys.dm_sql_referenced_entities

sys.dm_sql_referencing_entities
```

Types of Views

3D DIAGRAM

View Families



These are learning categories; SQL Server does not require a SIMPLE or COMPLEX keyword when creating a view.

SQL Server does not require you to label a view as "simple" or "complex" when creating it.

These are useful learning categories.

Common categories include:

EXAMPLE / FLOW

```
Simple views

Filtered views

Joined views

Aggregate views

GROUP BY views

HAVING views

UNION views

Views with subqueries

Security-oriented views

Updatable views

Schema-bound views

Indexed views

Nested views
```

Simple View

A simple view usually references one base table without complex calculations.

Example:

```sql
CREATE VIEW constituent.vw_ConstituentBasic
AS

SELECT
    ConstituentId,
    TenantId,
    DisplayName,
    ConstituentType,
    Status
FROM constituent.Constituent;
GO
```

Use this when you want a reusable subset of columns.

Filtered View

A filtered view exposes only rows satisfying a condition.

Example:

```sql
CREATE VIEW constituent.vw_ActiveConstituent
AS

SELECT
    ConstituentId,
    TenantId,
    DisplayName,
    ConstituentType,
    PrimaryEmail
FROM constituent.Constituent
WHERE Status = 'Active';
GO
```

Now:

```sql
SELECT *
FROM constituent.vw_ActiveConstituent;
```

returns only active Constituents.

Joined View

Simple Definition

A joined view combines data from multiple related tables.

Example:

```sql
CREATE VIEW fundraising.vw_GiftDetails
AS

SELECT
    g.GiftId,
    g.GiftDate,
    g.TotalAmount,

    c.ConstituentId,
    c.DisplayName AS ConstituentName,

    ca.CampaignId,
    ca.CampaignName

FROM fundraising.Gift AS g

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId

LEFT JOIN fundraising.Campaign AS ca
```

```
    ON ca.CampaignId = g.CampaignId;
GO
```

Why Joined Views Are Useful

Without the view, developers repeatedly write:

EXAMPLE / FLOW

```
Gift
JOIN Constituent
JOIN Campaign
```

The view standardizes that logic.

GiftAllocation Joined View

Another realistic example:

SQL

```
CREATE VIEW fundraising.vw_GiftAllocationDetails
AS

SELECT
    ga.GiftAllocationId,

    ga.GiftId,
    g.GiftDate,
    g.TotalAmount,

    ga.FundId,
    f.FundName,

    ga.AllocationAmount

FROM fundraising.GiftAllocation AS ga

INNER JOIN fundraising.Gift AS g
    ON g.GiftId = ga.GiftId

INNER JOIN fundraising.Fund AS f
    ON f.FundId = ga.FundId;
GO
```

The Gift/Fund many-to-many relationship in your real schema is implemented through `GiftAllocation`.

Aggregate View

Simple Definition

An aggregate view contains functions such as:

EXAMPLE / FLOW

```
COUNT

SUM

AVG

MIN

MAX
```

Example:

Lifetime giving by Constituent.

```sql
CREATE VIEW fundraising.vw_ConstituentGiving
AS

SELECT
    ConstituentId,

    COUNT(*) AS GiftCount,

    SUM(TotalAmount)
        AS LifetimeGiving,

    AVG(TotalAmount)
        AS AverageGift,

    MAX(TotalAmount)
        AS LargestGift

FROM fundraising.Gift

GROUP BY ConstituentId;
GO
```

Query Aggregate View

```sql
SELECT
    ConstituentId,
    GiftCount,
    LifetimeGiving,
    LargestGift
FROM fundraising.vw_ConstituentGiving

ORDER BY LifetimeGiving DESC;
```

Aggregate View with Constituent Name

```sql
CREATE VIEW fundraising.vw_ConstituentGivingSummary
AS

SELECT
    c.ConstituentId,
    c.DisplayName,

    COUNT(g.GiftId)
        AS GiftCount,

    SUM(g.TotalAmount)
        AS LifetimeGiving

FROM constituent.Constituent AS c

INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId

GROUP BY
    c.ConstituentId,
    c.DisplayName;
GO
```

View with GROUP BY

A view can contain **GROUP BY**.

Example:

Campaign fundraising totals.

```sql
CREATE VIEW fundraising.vw_CampaignGiving
AS

SELECT
    CampaignId,

    COUNT(*) AS GiftCount,

    COUNT(DISTINCT ConstituentId)
        AS DonorCount,

    SUM(TotalAmount)
        AS AmountRaised

FROM fundraising.Gift

WHERE CampaignId IS NOT NULL

GROUP BY CampaignId;
GO
```

View with HAVING

A normal view can also use `HAVING`.

Example:

Only Campaigns that have raised at least $100,000.

```sql
CREATE VIEW fundraising.vw_MajorCampaignGiving
AS

SELECT
    CampaignId,

    COUNT(*) AS GiftCount,

    SUM(TotalAmount)
        AS AmountRaised

FROM fundraising.Gift

WHERE CampaignId IS NOT NULL

GROUP BY CampaignId

HAVING SUM(TotalAmount) >= 100000;
GO
```

Important Indexed-View Difference

A normal view can contain `HAVING`.

But an **indexed view definition cannot contain `HAVING`**. If an indexed view uses `GROUP BY`, SQL Server imposes additional requirements such as `COUNT_BIG(*)`.

Do not confuse:

```
normal view rules
```

with:

EXAMPLE / FLOW

```
indexed view rules.
```

UNION View

A normal view can use set operators such as `UNION` or `UNION ALL`.

Example:

Build a combined Constituent activity view from Gifts and Interactions.

SQL

```
CREATE VIEW engagement.vw_ConstituentActivity
AS

SELECT
    ConstituentId,

    CAST(GiftDate AS DATETIME2)
        AS ActivityDate,

    'Gift'
        AS ActivityType

FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,

    CAST(InteractionDate AS DATETIME2),

    'Interaction'

FROM engagement.Interaction;
GO
```

Now:

SQL

```
SELECT *
FROM engagement.vw_ConstituentActivity;
```

returns both activity types.

Indexed View Restriction with UNION

An indexed view cannot contain:

EXAMPLE / FLOW

```
UNION

UNION ALL

INTERSECT

EXCEPT
```

even though ordinary views can.

View with Subquery

A normal view can also use subqueries.

Example:

Constituents whose total giving is greater than zero.

A correlated existence-style view:

```sql
CREATE VIEW constituent.vw_Donors
AS

SELECT
    c.ConstituentId,
    c.DisplayName,
    c.PrimaryEmail

FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId =
          c.ConstituentId
);
GO
```

Another View with Subquery

Find Constituents whose lifetime giving exceeds $10,000:

```sql
CREATE VIEW constituent.vw_MajorDonor
AS

SELECT
    c.ConstituentId,
    c.DisplayName

FROM constituent.Constituent AS c

WHERE
(
    SELECT SUM(g.TotalAmount)
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId =
          c.ConstituentId
) >= 10000;
GO
```

For complex reporting, however, an aggregate join or CTE may sometimes be easier to maintain and tune.

View with ORDER BY

A view should not be treated as having guaranteed order.

This is important:

A view represents a set of rows, not a permanently sorted result.

Normally write:

```sql
SELECT *
FROM fundraising.vw_ConstituentGiving

ORDER BY LifetimeGiving DESC;
```

in the outer query.

Even if special syntax allows `ORDER BY` with `TOP` inside a view, do not rely on a view definition to guarantee final output order.

Use `ORDER BY` in the consuming query.

Updatable Views

Some views can be used not only for:

EXAMPLE / FLOW

```
SELECT
```

but also:

EXAMPLE / FLOW

```
INSERT

UPDATE

DELETE.
```

These are commonly called:

EXAMPLE / FLOW

```
updatable views.
```

Simple Updatable View Example

SQL

```
CREATE VIEW constituent.vw_ConstituentContact
AS

SELECT
    ConstituentId,
    DisplayName,
    PrimaryEmail,
    PrimaryPhone

FROM constituent.Constituent;
GO
```

This view directly exposes columns from one base table.

UPDATE Through a View

SQL

```
UPDATE constituent.vw_ConstituentContact

SET PrimaryEmail =
    'john.smith@example.org'

WHERE ConstituentId = 1001;
```

SQL Server updates:

EXAMPLE / FLOW

```
constituent.Constituent
```

because the view maps directly to that table.

INSERT Through a View

Suppose a view exposes the columns necessary for a valid insert into its underlying table.

A simplified example:

```sql
CREATE VIEW fundraising.vw_BasicFund
AS

SELECT
    TenantId,
    FundCode,
    FundName,
    IsRestricted

FROM fundraising.Fund;
GO
```

Then, if all omitted base-table columns have defaults, allow NULL, or otherwise don't require explicit values, an insert may be possible:

```sql
INSERT INTO fundraising.vw_BasicFund
(
    TenantId,
    FundCode,
    FundName,
    IsRestricted
)
VALUES
(
    1,
    'GENERAL',
    'General Fund',
    0
);
```

The actual insertability depends on the full base-table definition and required columns.

DELETE Through a View

For a simple one-table view:

```sql
DELETE FROM constituent.vw_ConstituentContact
WHERE ConstituentId = 1001;
```

would attempt to delete that underlying Constituent row.

Whether it succeeds also depends on:

```
foreign keys,

other constraints,

triggers,

and business rules.
```

Conditions for an Updatable View

SQL Server can directly modify data through a view when the modification can be mapped unambiguously to underlying base-table rows.

Important rules include:

```
The modification must affect
columns from only one base table.

Modified columns must map directly
```

```
to base-table columns.

They cannot be aggregate results.

They cannot be calculated expressions.

GROUP BY / HAVING / DISTINCT
can prevent direct modification.

Set-operation-derived columns
are not directly updatable.
```

Microsoft documents these rules for updatable views.

Non-Updatable Aggregate View

This:

SQL

```
CREATE VIEW fundraising.vw_ConstituentGiving
AS

SELECT
    ConstituentId,
    SUM(TotalAmount)
        AS LifetimeGiving

FROM fundraising.Gift

GROUP BY ConstituentId;
```

produces:

EXAMPLE / FLOW

```
one result row
from many Gift rows.
```

What would this mean?

SQL

```
UPDATE fundraising.vw_ConstituentGiving
SET LifetimeGiving = 50000;
```

Which individual Gift rows should SQL Server change?

There is no unambiguous answer.

Therefore an aggregate result like this is not directly updateable.

Joined Views and Updates

A joined view may sometimes allow an update if SQL Server can clearly map the changed columns to one base table.

Example:

EXAMPLE / FLOW

```
Gift
JOIN
Constituent
```

If only a directly exposed `Gift` column is being updated, some joined views can permit the change.

But you cannot issue one direct modification that ambiguously changes multiple base tables.

Microsoft's rule is that modifications through a view must reference columns from only one base table.

INSTEAD OF Triggers and Views

When a view is too complex for normal direct updates, SQL Server supports:

EXAMPLE / FLOW

```
INSTEAD OF triggers
```

on views.

An `INSTEAD OF` trigger can define what should happen when a user attempts:

EXAMPLE / FLOW

```
INSERT

UPDATE

DELETE
```

through the view.

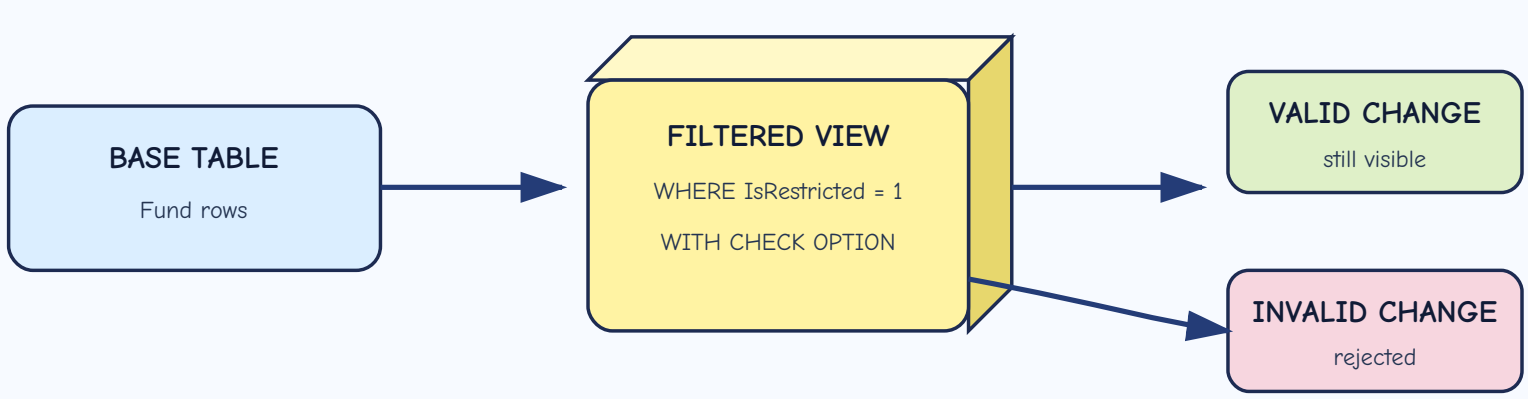
This is an advanced trigger topic, but it is important to know that it can make otherwise non-directly-updatable views support controlled modifications.

WITH CHECK OPTION

3D DIAGRAM

Filtered View + CHECK OPTION

Rows modified through the filtered view must continue to satisfy the view filter.



CHECK OPTION governs modifications through the view; direct base-table changes are a separate path.

Simple Definition

`WITH CHECK OPTION` ensures that rows inserted or updated **through the view** must still satisfy the view's filtering condition afterward.

This is easiest to understand with **Fund**.

Restricted Funds View

Suppose:

SQL

```
CREATE VIEW fundraising.vw_RestrictedFund
AS
```

```
SELECT
    FundId,
    TenantId,
    FundCode,
    FundName,
    IsRestricted

FROM fundraising.Fund

WHERE IsRestricted = 1

WITH CHECK OPTION;
GO
```

The view only contains:

EXAMPLE / FLOW

```
restricted Funds.
```

Update Without CHECK OPTION Problem

Without `WITH CHECK OPTION`, someone might update:

SQL

```
UPDATE fundraising.vw_RestrictedFund
SET IsRestricted = 0
WHERE FundId = 300;
```

The row would then no longer meet:

EXAMPLE / FLOW

```
IsRestricted = 1
```

and would disappear from the view.

With CHECK OPTION

Because the view contains:

SQL

```
WITH CHECK OPTION
```

SQL Server rejects a modification through the view that would make the row fail the view condition.

Microsoft states that `CHECK OPTION` applies to modifications performed through the view; direct changes to the underlying base table are not governed by that view's `CHECK OPTION`.

Important CHECK OPTION Memory Rule

EXAMPLE / FLOW

```
View filter
+
WITH CHECK OPTION
=
Rows modified through the view
must remain visible through the view.
```

Security Through Views

Views can be useful as a controlled interface over tables.

Two common security-related uses are:

EXAMPLE / FLOW

```
Column-level exposure

Row-level filtering
```

Column-Level Exposure

Your `Constituent` table may contain:

EXAMPLE / FLOW

```
DisplayName

PrimaryEmail

PrimaryPhone

other internal fields
```

Suppose a reporting user needs only:

EXAMPLE / FLOW

```
ConstituentId

DisplayName

ConstituentType
```

Create:

SQL

```
CREATE VIEW constituent.vw_ConstituentPublic
AS

SELECT
    ConstituentId,
    DisplayName,
    ConstituentType

FROM constituent.Constituent;
GO
```

Now sensitive or unnecessary columns are not exposed by that view.

Grant Permission on View

Example:

SQL

```
GRANT SELECT
ON OBJECT::constituent.vw_ConstituentPublic
TO ReportingUser;
```

The user can be given access to the view rather than broad direct table access.

Actual security behavior also depends on ownership chains and permissions, so security design should be deliberate.

Hiding Sensitive Columns

Suppose normal reports should not expose:

EXAMPLE / FLOW

```
PrimaryEmail

PrimaryPhone
```

Do not include them in the public view.

This does not remove those columns from the base table.

It changes the interface exposed to that consumer.

Row-Level Filtering Through a View

Example:

SQL

```
CREATE VIEW constituent.vw_ActiveConstituent
AS

SELECT
    ConstituentId,
    TenantId,
    DisplayName,
    ConstituentType

FROM constituent.Constituent

WHERE Status = 'Active';
GO
```

Users of this view see only:

EXAMPLE / FLOW

```
Active Constituents.
```

Tenant Filtering Through Views

Because your database is multi-tenant, every tenant owns its own business data.

A hard-coded teaching example might be:

SQL

```
CREATE VIEW constituent.vw_Tenant1Constituent
AS

SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent
WHERE TenantId = 1;
```

But this is **not a scalable security model** for a SaaS application.

You would not create:

EXAMPLE / FLOW

```
one view per tenant.
```

For serious per-user/per-tenant security, SQL Server Row-Level Security or application-enforced tenant boundaries are generally more appropriate.

Use views for appropriate data exposure, not as the only multi-tenant security mechanism.

View vs Table

View	Table
Stores query definition	Stores rows physically
Usually virtual result	Physical relational object
Can join many tables	Represents stored entity/data
Can filter/hide columns	Holds actual data

View	Table
Some views updateable	Tables directly updateable subject to constraints
Normal view has no independent row storage	Table owns its rows

Easy memory:

EXAMPLE / FLOW

```
Table
→ stores data

View
→ stores a query over data
```

View vs Temporary Table

View	#Temp Table
Persistent database object	Temporary object
Stores query definition	Stores materialized rows
Reusable across sessions if permitted	Usually session-scoped
Cannot be populated manually like temp table	Can insert/update temp rows
Normal view has no own indexes	Temp table can have indexes
Good for reusable logical abstraction	Good for staged intermediate processing

View vs CTE

View	CTE
Persistent object	One-statement scope
Stored in database metadata	Defined inside a statement
Reusable	Used by immediate statement
Can receive permissions	No separate securable object
Good shared abstraction	Good query-specific organization

View vs Stored Procedure

View	Stored Procedure
Represents a query result	Executable program/module
Used in `FROM` like a table	Executed with `EXEC`
No ordinary parameters	Can accept parameters
Usually declarative SELECT abstraction	Can contain procedural logic
Can be joined to other queries	Result sets aren't used like normal tables
Useful for reusable data shape	Useful for operations/workflows

Example

View:

```sql
SELECT *
FROM fundraising.vw_ConstituentGiving;
```

Stored procedure:

```sql
EXEC fundraising.usp_GetConstituentGiving
    @ConstituentId = 1001;
```

The stored procedure can accept parameters and execute more complex logic.

A normal view cannot be parameterized.

Nested Views

A nested view is a view that references another view.

Example:

```sql
CREATE VIEW fundraising.vw_GiftBase
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift;
GO
```

Then:

```sql
CREATE VIEW fundraising.vw_LargeGift
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM fundraising.vw_GiftBase

WHERE TotalAmount >= 1000;
GO
```

`vw_LargeGift` depends on:

```
EXAMPLE / FLOW
vw_GiftBase
```

which depends on:

```
EXAMPLE / FLOW
Gift.
```

Are Nested Views Bad?

Not automatically.

But excessive nesting can make queries difficult to:

EXAMPLE / FLOW

```
understand,

debug,

trace,

and tune.
```

Example:

EXAMPLE / FLOW

```
View A
 ↓
View B
 ↓
View C
 ↓
View D
 ↓
Many base tables
```

A developer querying View D may not realize how much work is happening underneath.

View Performance

Important Rule

A normal view is not automatically faster than writing the equivalent query directly.

Example:

SQL

```
SELECT *
FROM fundraising.vw_GiftDonor;
```

and the equivalent underlying join can often produce similar optimized plans.

The optimizer expands the normal view definition into the larger query.

What Determines View Performance?

Important factors include:

EXAMPLE / FLOW

```
base-table indexes,

statistics,

join conditions,

filter selectivity,

aggregations,

row counts,

nested-view complexity,

data types,

execution plan.
```

A View Is Not a Cache

A common misunderstanding:

"If I create a view, SQL Server stores the result and the query becomes fast."

For a **normal view**, this is generally false.

It stores the:

EXAMPLE / FLOW

```
query definition
```

rather than a permanently materialized result.

Indexed Views Are Different

An indexed view is a special view whose result is physically persisted through an index.

This is very different from a normal view.

WITH SCHEMABINDING

Simple Definition

`WITH SCHEMABINDING` binds the view to the schema of the objects it references.

Example:

SQL

```
CREATE VIEW fundraising.vw_SchemaBoundGift
WITH SCHEMABINDING
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM fundraising.Gift;
GO
```

What Does SCHEMABINDING Do?

If a view is schema-bound, SQL Server prevents changes to underlying objects that would break the view.

For example, you cannot casually drop:

EXAMPLE / FLOW

```
Gift.TotalAmount
```

while a schema-bound view depends on that column.

The view must first be changed or removed.

Microsoft documents that schema-bound referenced tables cannot be changed in ways that affect the view definition until the binding is removed or the view is changed.

SCHEMABINDING Naming Rule

Schema-bound objects must use two-part names such as:

EXAMPLE / FLOW

```
fundraising.Gift
```

not simply:

EXAMPLE / FLOW

```
Gift.
```

Referenced objects must also be in the same database.

Why Use SCHEMABINDING?

Common reasons:

EXAMPLE / FLOW

```
Protect view dependencies

Prevent breaking schema changes

Required for indexed views
```

WITH ENCRYPTION

SQL Server supports:

SQL

```
WITH ENCRYPTION
```

on a view definition.

Example:

SQL

```
CREATE VIEW fundraising.vw_EncryptedGift
WITH ENCRYPTION
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount

FROM fundraising.Gift;
GO
```

Microsoft documents that this hides the stored view definition from normal metadata exposure and prevents the definition from being published through SQL Server replication.

Important ENCRYPTION Warning

Do not think:

EXAMPLE / FLOW

```
WITH ENCRYPTION
=
strong data encryption.
```

It does not encrypt the underlying Gift rows.

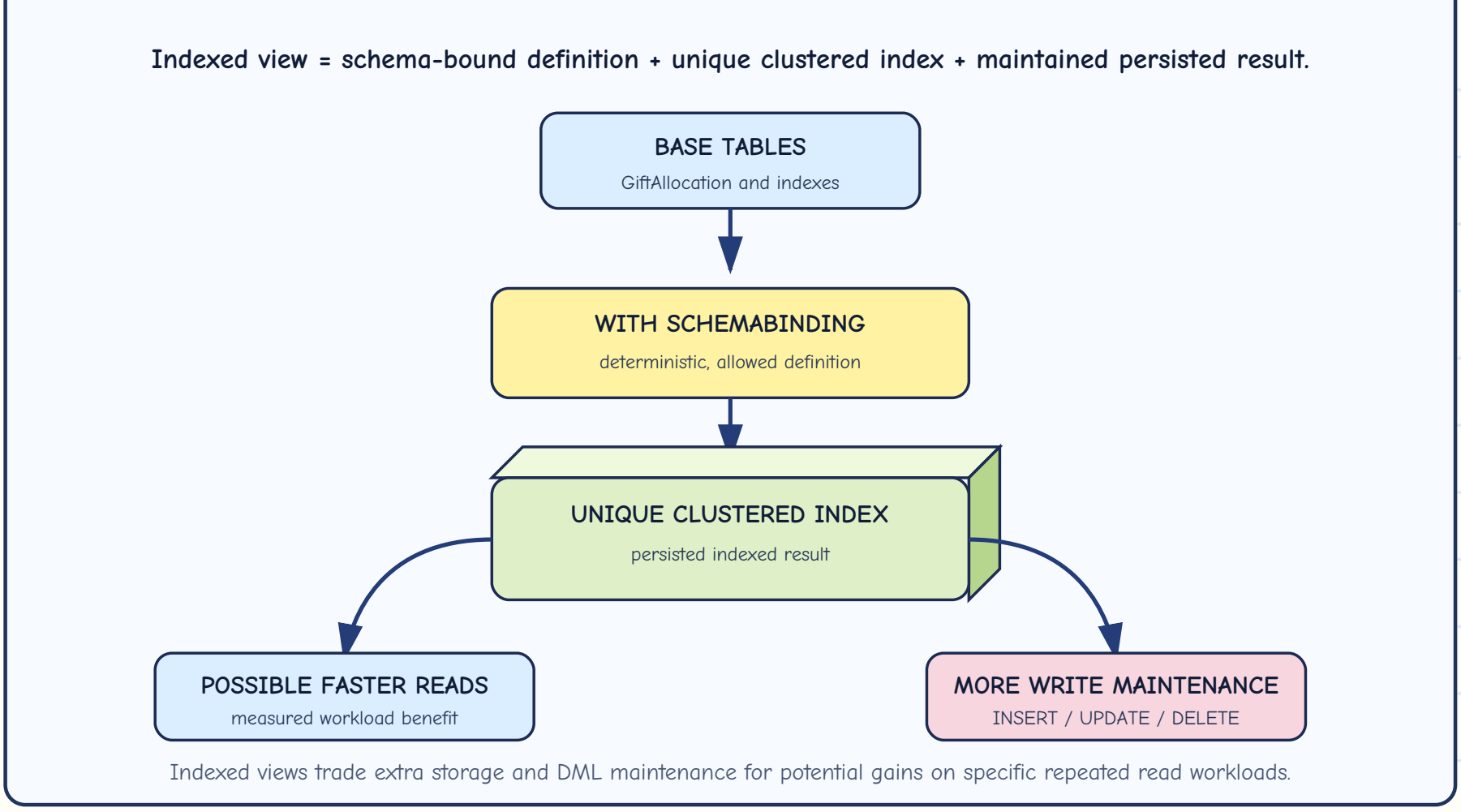
It mainly obscures the stored module definition.

Because it makes maintenance and source recovery harder, keep the view definition safely stored in source control.

Indexed Views

3D DIAGRAM

Indexed View Pipeline



Simple Definition

An **indexed view** is a view on which SQL Server creates indexes.

Once the required unique clustered index exists, the view result is physically stored at the index leaf level and maintained as underlying data changes.

This is very different from a normal virtual view.

Why Use an Indexed View?

Indexed views can help certain workloads where the same expensive query or aggregation is repeatedly used.

Typical candidate:

EXAMPLE / FLOW

```
large table

repeated aggregation

mostly read-oriented workload

expensive calculation
```

Fundraising Indexed View Scenario

Suppose `GiftAllocation` eventually contains hundreds of millions of rows.

Reports constantly need:

EXAMPLE / FLOW

```
total allocation amount
per Fund.
```

Normal query:

SQL

```
SELECT
    FundId,
    COUNT_BIG(*) AS AllocationCount,
    SUM(AllocationAmount) AS TotalAllocated

FROM fundraising.GiftAllocation

GROUP BY FundId;
```

This could become a candidate for an indexed-view discussion.

Indexed View Creation Requirements

The important high-level sequence is:

EXAMPLE / FLOW

```
1. Required SET options

2. Deterministic view definition

3. Compatible ownership

4. WITH SCHEMABINDING

5. Create unique clustered index

6. Optionally add nonclustered indexes
```

These are explicit SQL Server indexed-view requirements.

Required SET Options

Indexed views require specific session settings, including:

EXAMPLE / FLOW

```
ANSI_NULLS ON

ANSI_PADDING ON

ANSI_WARNINGS ON

ARITHABORT ON

CONCAT_NULL_YIELDS_NULL ON

QUOTED_IDENTIFIER ON

NUMERIC_ROUNDABORT OFF
```

These settings must be correct when creating and maintaining indexed views.

Indexed View Example

A simplified learning example:

SQL

```
SET NUMERIC_ROUNDABORT OFF;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET ARITHABORT ON;
SET QUOTED_IDENTIFIER ON;
```

```
SET ANSI_NULLS ON;
GO
```

Create the view:

SQL

```
CREATE VIEW fundraising.vw_FundAllocationSummary
WITH SCHEMABINDING
AS

SELECT
    FundId,

    COUNT_BIG(*)
        AS AllocationCount,

    SUM(AllocationAmount)
        AS TotalAllocated

FROM fundraising.GiftAllocation

GROUP BY FundId;
GO
```

Why COUNT_BIG?

For indexed views that contain **GROUP BY**, SQL Server requires:

SQL

```
COUNT_BIG(*)
```

and does not allow ordinary **COUNT(*)** as the indexed-view count expression.

Create Unique Clustered Index

The first index on an indexed view must be:

EXAMPLE / FLOW

```
UNIQUE CLUSTERED
```

Example:

SQL

```
CREATE UNIQUE CLUSTERED INDEX
    IX_vw_FundAllocationSummary_FundId

ON fundraising.vw_FundAllocationSummary
(
    FundId
);
GO
```

Microsoft requires the first index on a view to be a unique clustered index.

Why Must It Be Unique?

SQL Server needs a key that uniquely identifies each persisted indexed-view row.

Because the view groups by:

EXAMPLE / FLOW

```
FundId
```

each FundId produces one row.

Therefore:

EXAMPLE / FLOW

```
FundId
```

can be the unique clustered index key.

Additional Indexes on Indexed Views

After the unique clustered index exists, you may add:

EXAMPLE / FLOW

```
nonclustered indexes
```

to support other query patterns.

Example:

SQL

```
CREATE NONCLUSTERED INDEX
    IX_vw_FundAllocationSummary_TotalAllocated

ON fundraising.vw_FundAllocationSummary
(
    TotalAllocated
);
```

Whether this particular index is beneficial must be based on actual query workload.

Indexed View Determinism

An indexed view must use deterministic expressions.

A deterministic expression returns the same result for the same input.

Example:

EXAMPLE / FLOW

```
DATEADD
```

can be deterministic.

But:

EXAMPLE / FLOW

```
GETDATE()
```

is nondeterministic because time changes between calls.

SQL Server requires deterministic view definitions for indexed views.

Indexed View Restrictions

Indexed views have many more restrictions than normal views.

Examples of constructs not allowed in an indexed-view definition include:

EXAMPLE / FLOW

```
CTEs

subqueries

SELECT *

DISTINCT

TOP
```

```
window functions / OVER

LEFT / RIGHT / FULL OUTER JOIN

APPLY

self joins

derived tables

UNION / UNION ALL

INTERSECT / EXCEPT

HAVING

ORDER BY

GROUPING SETS / ROLLUP / CUBE
```

There are also aggregate and data-type restrictions.

Important Difference

This normal view is valid conceptually:

SQL

```
CREATE VIEW engagement.vw_Activity
AS

SELECT ...
FROM fundraising.Gift

UNION ALL

SELECT ...
FROM engagement.Interaction;
```

But:

EXAMPLE / FLOW

```
you cannot simply turn it
into an indexed view.
```

`UNION ALL` is prohibited in indexed-view definitions.

Indexed View Maintenance Cost

This is one of the most important concepts.

Suppose:

EXAMPLE / FLOW

```
vw_FundAllocationSummary
```

is indexed.

Now insert:

SQL

```
INSERT INTO fundraising.GiftAllocation
(
    TenantId,
    GiftId,
    FundId,
```

```
    AllocationAmount
)
VALUES
(
    1,
    10000,
    300,
    700
);
```

SQL Server must maintain:

EXAMPLE / FLOW

```
GiftAllocation base-table indexes
+
indexed-view data/indexes.
```

So reads may become faster, but writes become more expensive.

Microsoft warns that INSERT/UPDATE/DELETE performance can degrade when base tables participate in indexed views because those views must also be maintained.

Indexed View Performance Trade-Off

EXAMPLE / FLOW

```
Possible faster reads
        ↕
More expensive writes
```

This is particularly important because `FundraisingDb` is an:

EXAMPLE / FLOW

```
OLTP database.
```

Your database is explicitly designed for day-to-day fundraising transactions.

Therefore do not add indexed views everywhere.

Measure first.

When to Consider Indexed Views

Potential candidate:

EXAMPLE / FLOW

```
Large base tables

Repeated expensive aggregation

Stable query pattern

Read-heavy reporting

Significant measured benefit
```

When Not to Use Indexed Views

Avoid them blindly when:

EXAMPLE / FLOW

```
tables have heavy DML,
```

```
view is complex,

normal indexing already solves performance,

data volume is small,

or the query is rarely used.
```

Normal View vs Indexed View

Normal View	Indexed View
Stores definition	Definition + persisted indexed result
Usually no own row storage	Result physically represented by index
Few special definition restrictions	Many strict restrictions
No unique clustered index required	Unique clustered index required first
Low base-table write overhead from view itself	DML must maintain indexed view
Mainly abstraction/security/reuse	Primarily special performance optimization

sp_refreshview

Normal non-schema-bound views can sometimes have metadata that becomes stale after changes to underlying objects.

SQL Server provides:

```sql
sys.sp_refreshview
```

Example:

```sql
EXEC sys.sp_refreshview
    N'fundraising.vw_GiftDonor';
```

Microsoft recommends refreshing a non-schema-bound view when underlying object changes affect the view definition, otherwise the view can return unexpected metadata/results.

Example Metadata Problem

Suppose a non-schema-bound view uses:

```sql
SELECT *
FROM fundraising.Gift;
```

Then the base table changes.

Using **SELECT *** in persistent modules can make metadata harder to reason about.

Better:

```sql
SELECT
    GiftId,
    TenantId,
    ConstituentId,
    CampaignId,
    GiftDate,
```

```
    TotalAmount
FROM fundraising.Gift;
```

Explicit columns make dependencies clearer.

Best Practice — Avoid SELECT * in Views

Less ideal:

SQL

```
CREATE VIEW fundraising.vw_Gift
AS
SELECT *
FROM fundraising.Gift;
```

Better:

SQL

```
CREATE VIEW fundraising.vw_Gift
AS

SELECT
    GiftId,
    TenantId,
    ConstituentId,
    CampaignId,
    GiftDate,
    TotalAmount,
    CurrencyCode,
    GiftType
FROM fundraising.Gift;
```

Advantages:

EXAMPLE / FLOW

```
clear contract,

better dependency visibility,

less accidental exposure,

safer schema changes.
```

Checking View Dependencies Before Table Changes

Before changing:

EXAMPLE / FLOW

```
fundraising.Gift
```

check which objects depend on it:

SQL

```
SELECT
    OBJECT_SCHEMA_NAME(referencing_id)
        AS SchemaName,

    OBJECT_NAME(referencing_id)
        AS ObjectName

FROM sys.sql_expression_dependencies

WHERE referenced_id =
      OBJECT_ID
      (
```

```
        'fundraising.Gift'
    );
```

Dependency checking is particularly useful before:

EXAMPLE / FLOW

```
renaming columns,

dropping columns,

changing data types,

dropping tables.
```

Views with SCHEMABINDING vs sp_refreshview

Schema-bound view:

EXAMPLE / FLOW

```
SQL Server blocks incompatible
underlying schema changes.
```

Non-schema-bound view:

EXAMPLE / FLOW

```
Underlying schema can change,
so view metadata may need refreshing.
```

That is why:

EXAMPLE / FLOW

```
SCHEMABINDING
```

and:

EXAMPLE / FLOW

```
sp_refreshview
```

solve related but different problems.

Security Best Practices with Views

Views can help reduce exposure, but do not treat them as a complete security strategy by themselves.

Use views together with:

EXAMPLE / FLOW

```
least-privilege permissions,

appropriate schemas,

ownership-chain understanding,

Row-Level Security where needed,

application authorization,

auditing.
```

Common View Mistakes

Mistake 1 — Thinking a View Stores Data

Normal view:

EXAMPLE / FLOW

```
stores query definition.
```

Indexed view:

EXAMPLE / FLOW

```
has physically persisted indexed result.
```

Do not confuse them.

Mistake 2 — Creating a View for Every Query

A view should represent:

EXAMPLE / FLOW

```
reusable logic,

useful abstraction,

security boundary,

or stable reporting shape.
```

Do not create hundreds of views for one-off queries.

Mistake 3 — Using SELECT *

Avoid:

SQL

```
SELECT *
```

in long-lived view definitions.

Explicitly select columns.

Mistake 4 — Assuming Views Are Automatically Faster

Normal views generally do not cache results.

Performance comes mainly from:

EXAMPLE / FLOW

```
query design

indexes

statistics

and optimizer choices.
```

Mistake 5 — Deep Nested Views

Avoid:

EXAMPLE / FLOW

```
View A
→ View B
→ View C
→ View D
→ View E
```

unless there is a strong architectural reason.

Deep nesting hides complexity.

Mistake 6 — Treating View Security as Absolute

Hiding a column in one view does not magically make the base table secure.

You must also manage:

EXAMPLE / FLOW

```
permissions.
```

Mistake 7 — Using WITH ENCRYPTION as Data Security

`WITH ENCRYPTION` does not encrypt underlying fundraising data.

It obscures the module definition.

Mistake 8 — Forgetting WITH CHECK OPTION

If the view is intended to enforce its filter for updates performed through the view, use:

SQL

```
WITH CHECK OPTION
```

when appropriate.

Mistake 9 — Updating Aggregate Views

This makes no logical sense:

EXAMPLE / FLOW

```
LifetimeGiving = 50000
```

because the value came from many Gift rows.

Aggregate views are normally read-oriented.

Mistake 10 — Adding Indexed Views Before Measuring

Indexed views add DML maintenance cost.

Use execution plans and workload testing first.

Practical View 1 — Donor Gift Summary

SQL

```
CREATE VIEW fundraising.vw_DonorGiftSummary
AS

SELECT
    c.ConstituentId,
    c.DisplayName,

    COUNT(g.GiftId)
        AS GiftCount,

    SUM(g.TotalAmount)
        AS LifetimeGiving,

    MAX(g.TotalAmount)
        AS LargestGift,

    MAX(g.GiftDate)
        AS LatestGiftDate
```

```
FROM constituent.Constituent AS c

INNER JOIN fundraising.Gift AS g
    ON g.ConstituentId = c.ConstituentId

GROUP BY
    c.ConstituentId,
    c.DisplayName;
GO
```

This could support:

EXAMPLE / FLOW

```
donor reports,

dashboard pages,

fundraiser summaries.
```

Practical View 2 — Gift Allocation Detail

SQL

```
CREATE VIEW fundraising.vw_GiftAllocationDetail
AS

SELECT
    g.GiftId,
    g.GiftDate,

    c.ConstituentId,
    c.DisplayName,

    f.FundId,
    f.FundName,

    ga.AllocationAmount

FROM fundraising.GiftAllocation AS ga

INNER JOIN fundraising.Gift AS g
    ON g.GiftId = ga.GiftId

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId =
       g.ConstituentId

INNER JOIN fundraising.Fund AS f
    ON f.FundId = ga.FundId;
GO
```

Practical View 3 — Open Tasks

SQL

```
CREATE VIEW engagement.vw_OpenTask
AS

SELECT
    TaskId,
    TenantId,
    ConstituentId,
    AssignedToUserId,
    CreatedByUserId,
    Title,
    DueDate,
    Priority

FROM engagement.Task
```

```
WHERE CompletedAtUtc IS NULL;
GO
```

Now:

```sql
SELECT *
FROM engagement.vw_OpenTask;
```

shows incomplete Tasks.

Practical View 4 — AppUser Task Workload

```sql
CREATE VIEW engagement.vw_AppUserOpenTaskCount
AS

SELECT
    AssignedToUserId AS AppUserId,

    COUNT(*) AS OpenTaskCount

FROM engagement.Task

WHERE CompletedAtUtc IS NULL

GROUP BY AssignedToUserId;
GO
```

Practical View 5 — Constituent Activity

```sql
CREATE VIEW engagement.vw_ConstituentActivity
AS

SELECT
    ConstituentId,

    CAST(GiftDate AS DATETIME2)
        AS ActivityDate,

    'Gift'
        AS ActivityType

FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,

    CAST(InteractionDate AS DATETIME2),

    'Interaction'

FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,

    CAST(DueDate AS DATETIME2),

    'Task'
```

```
FROM engagement.Task;
GO
```

Query it with final ordering:

SQL

```
SELECT *
FROM engagement.vw_ConstituentActivity

ORDER BY ActivityDate DESC;
```

Practical View 6 — Limited Public Constituent Exposure

SQL

```
CREATE VIEW constituent.vw_ConstituentDirectory
AS

SELECT
    ConstituentId,
    DisplayName,
    ConstituentType,
    Status

FROM constituent.Constituent;
GO
```

Notice that this intentionally omits:

EXAMPLE / FLOW

```
PrimaryEmail

PrimaryPhone
```

if those are not appropriate for that consumer.

Choosing a View

Use a view when:

EXAMPLE / FLOW

```
query logic is reused,

a stable logical data shape is useful,

you want to hide unnecessary columns,

you want common joins in one place,

you need a reusable reporting abstraction.
```

Do Not Use a View When

A view may not be appropriate when:

EXAMPLE / FLOW

```
logic is used once,

parameters are required,

procedural steps are required,
```

```
temporary materialization is required,

complex intermediate indexes are needed.
```

Choose:

EXAMPLE / FLOW

```
CTE

#Temp table

Stored procedure

normal query
```

when they fit better.

Master View Comparison

EXAMPLE / FLOW

```
TABLE
→ Stores rows
```

EXAMPLE / FLOW

```
VIEW
→ Stores reusable query definition
```

EXAMPLE / FLOW

```
CTE
→ Names query logic for one statement
```

EXAMPLE / FLOW

```
#TEMP TABLE
→ Stores temporary rows
```

EXAMPLE / FLOW

```
STORED PROCEDURE
→ Executes reusable parameterized logic
```

EXAMPLE / FLOW

```
INDEXED VIEW
→ Persisted indexed result
```

25 Interview Questions and Practical Problems

1. What is a view in SQL Server?

A view is a named query definition stored in the database.

A normal view behaves like a virtual table but normally obtains its rows from underlying tables when queried.

2. Why do we use views?

Views are useful for:

EXAMPLE / FLOW

```
simplifying complex joins,

reusing query logic,
```

```
hiding columns,

filtering rows,

standardizing reports,

and controlling data exposure.
```

3. Does a normal view store data physically?

Normally no.

It stores the query definition.

An indexed view is different because an index physically stores the indexed result.

4. How do you create a view?

SQL

```
CREATE VIEW fundraising.vw_LargeGift
AS

SELECT
    GiftId,
    ConstituentId,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

5. What is ALTER VIEW?

`ALTER VIEW` changes the definition of an existing view without dropping the object first.

Example:

SQL

```
ALTER VIEW fundraising.vw_LargeGift
AS

SELECT
    GiftId,
    ConstituentId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TotalAmount >= 1000;
```

6. What does DROP VIEW do?

It removes the view definition.

It does not delete underlying table data.

SQL

```
DROP VIEW fundraising.vw_LargeGift;
```

7. Can a view join multiple tables?

Yes.

Example:

```sql
SELECT
    g.GiftId,
    c.DisplayName
FROM fundraising.Gift AS g

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId =
       g.ConstituentId;
```

can be stored as a joined view.

8. Can a view contain GROUP BY and aggregate functions?

Yes, a normal view can contain:

```
GROUP BY

COUNT

SUM

AVG

MIN

MAX
```

Example: lifetime giving per Constituent.

Such aggregate views are generally not directly updatable because each view row may represent multiple base rows.

9. Can a view contain HAVING?

A normal view can.

Example:

```sql
GROUP BY CampaignId
HAVING SUM(TotalAmount) >= 100000
```

However, indexed-view definitions cannot contain `HAVING`.

10. Can a view contain UNION?

A normal view can use set operators such as `UNION` and `UNION ALL`.

Indexed views cannot contain `UNION`, `UNION ALL`, `INTERSECT`, or `EXCEPT`.

11. What is an updatable view?

An updatable view is a view through which SQL Server can map a modification back to underlying base-table rows.

Simple one-table views are the easiest examples.

12. What are important restrictions for updating through a view?

The modification must generally be unambiguous and affect one base table.

Aggregate values, calculated columns, `GROUP BY`, `HAVING`, `DISTINCT`, and set-derived values can prevent direct updates.

13. Can you INSERT through a view?

Yes, when the view is updateable and enough required base-table columns are supplied.

The insert must be mappable to one base table, and omitted required columns must be otherwise satisfied.

14. Can you DELETE through a view?

Yes for suitable updateable views.

For a simple one-table view, deleting a view row deletes the underlying base-table row.

Foreign keys and other constraints still apply.

15. What is WITH CHECK OPTION?

`WITH CHECK OPTION` prevents an INSERT or UPDATE performed through a filtered view from leaving the row outside the view's filter condition.

Example:

EXAMPLE / FLOW

```
Restricted Funds view
WHERE IsRestricted = 1
```

With `CHECK OPTION`, you cannot update `IsRestricted` to `0` through that view.

16. What is WITH SCHEMABINDING?

`WITH SCHEMABINDING` binds the view to referenced objects and prevents underlying schema changes that would break the view.

It is also required for indexed views.

17. What is WITH ENCRYPTION?

`WITH ENCRYPTION` obscures the stored view definition from ordinary metadata display.

It does not encrypt the underlying table data.

Keep source definitions in source control before using it.

18. What is an indexed view?

An indexed view is a view whose result is persisted through indexes.

The first index must be a:

EXAMPLE / FLOW

```
UNIQUE CLUSTERED INDEX.
```

19. Why use indexed views?

They can improve performance for some expensive, repeated queries—particularly aggregations.

But they increase storage and DML maintenance cost because base-table changes must maintain the indexed view.

20. What is the main performance disadvantage of an indexed view?

Every relevant:

EXAMPLE / FLOW

```
INSERT

UPDATE

DELETE
```

on the underlying tables may also require indexed-view maintenance.

This can hurt transactional write performance.

21. What is the difference between a view and a CTE?

A view is a persistent database object reusable by many queries.

A CTE exists only for the immediately following statement.

22. What is the difference between a view and a temp table?

A view stores query logic.

A temp table materializes temporary rows in `tempdb` and can have its own indexes.

23. What is the difference between a view and a stored procedure?

A view represents a queryable relational result and does not take normal input parameters.

A stored procedure can accept parameters and execute procedural, multi-statement logic.

24. How do you find dependencies of a view?

Use modern SQL Server dependency metadata such as:

SQL

```
SELECT *
FROM sys.sql_expression_dependencies
WHERE referencing_id =
      OBJECT_ID
      (
          'fundraising.vw_GiftDonor'
      );
```

You can also use:

EXAMPLE / FLOW

```
sys.dm_sql_referenced_entities

sys.dm_sql_referencing_entities.
```

Avoid building new solutions around deprecated `sp_depends`.

25. What does sp_refreshview do?

`sp_refreshview` refreshes metadata for a non-schema-bound view after underlying object changes that affect the view.

Example:

SQL

```
EXEC sys.sp_refreshview
    N'fundraising.vw_GiftDonor';
```

This is especially relevant after changing columns used by a non-schema-bound view.

Final Revision Diagram

EXAMPLE / FLOW

```
BASE TABLES
     |
     +---- Constituent
     |
     +---- Gift
     |
     +---- Campaign
     |
     +---- Fund
```

```
     |
     +---- GiftAllocation
     |
     v

NORMAL VIEW
     |
     | Stores query definition
     |
     +---- Simple view
     +---- Filtered view
     +---- Joined view
     +---- Aggregate view
     +---- UNION view
     +---- Security view
     |
     v

APPLICATION / REPORT
```

Indexed view:

EXAMPLE / FLOW

```
Base tables
    |
    v
WITH SCHEMABINDING
    |
    v
View
    |
    v
UNIQUE CLUSTERED INDEX
    |
    v
Persisted indexed result
    |
    +---- Possible faster reads
    |
    +---- More DML maintenance
```

Final Easy Memory Rules

EXAMPLE / FLOW

```
View
→ Saved reusable SELECT definition
```

EXAMPLE / FLOW

```
Normal view
→ Usually does not store rows
```

EXAMPLE / FLOW

```
Joined view
→ Reusable joins
```

EXAMPLE / FLOW

```
Aggregate view
→ Reusable summaries
```

EXAMPLE / FLOW

```
WITH CHECK OPTION
→ Updates through filtered view
  must remain visible
```

EXAMPLE / FLOW

```
SCHEMABINDING
→ Protect underlying schema dependency
```

EXAMPLE / FLOW

```
Indexed view
→ Schema-bound view
  + unique clustered index
```

EXAMPLE / FLOW

```
WITH ENCRYPTION
→ Hides module definition,
  not underlying data
```

EXAMPLE / FLOW

```
sp_refreshview
→ Refresh metadata for
  non-schema-bound view
```

The most important overall rule is:



Use a view when you need a reusable, stable relational interface over your tables. Use it to simplify common joins, expose only appropriate columns and rows, standardize reporting logic, or provide reusable aggregates. Do not assume a normal view improves performance automatically, and use indexed views only after measuring a genuine performance benefit because their persisted indexes increase write-maintenance cost.